New Magic Tricks for Fault-Tolerant Quantum computation

Julio Magdalena de la Fuente

Scribe: Keshav Deoskar*

This set of notes is essentially a trasncription of Julio's talk on October 16 2026 as part of the University of Sydney's *"Quantum Fridays"* seminar, with some extra details/remarks added in (and likely many left-out because of my poor memory). We assume familiarity with the basics of the Stabilizer and CSS Formalisms (including the interpretation in terms of Chain Complexes), as well as the Toric Code.

Any errors are likely due to me being a silly goose. Corrections would be appreciated! This LaTeXtemplate is based Kevin Zhou's tex template.

*keshav.deoskar@uni.sydney.edu

Contents

1 Why Magic? 2
- 1.1 Clifford Hierarchy . . . 2
- 1.2 Universal Gatesets . . . 2

2 Magic in Codes 3
- 2.1 Fault-Tolerance and Transversality . . . 3
- 2.2 Gate Injection . . . 5
- 2.3 Common ways to prepare Magic . . . 6
  - Magic State Distillation . . . 6
  - Code Switching . . . 7

3 New kid on the block: Non-Pauli Codes 7
- Non-Pauli Codes . . . 7
- Eg. Preparing $|T\rangle$ state . . . 8
- Fault-Tolerance . . . 8

4 Algebra of Twisted CSS Codes (By Example) 9
- Commutation Relations of Stabilizers . . . 10

5 Future directions 11

---

1 Why Magic?

Quantum Computing is touted for its beyond-classical runtimes for certain types of problems i.e. it is interesting because we are able to achieve far far better runtimes/complexities for algorithms solving certain classes of problems than would be achieved in the classical case.

However, what exactly is it that gives us this beyond-classical capacility? To answer this question, we must discuss the *Clifford Hierarchy*.

1.1 Clifford Hierarchy

Recall that in the Stabilizer Formalism it is sufficient to consider Pauli Errors and how they (anti)commute with the stabilizers. Recall that the set of possible Pauli (X,Y,Z) errors acting on a set of $n-$qubits forms the **Pauli Group, $\mathcal{P}_n$**, which we will suggestively also denote $\mathcal{C}_0$.

$$\mathcal{C}_0 := \mathcal{P}_n = \langle \{X_j, Z_j\}_{1 \leq j \leq n}, i \rangle$$

Another important set of gates is that of **Clifford Gates**, which map Paulis to Paulis under conjugation i.e. this is the Normalizer of the Pauli group.

$$\mathcal{C}_1 := \{u \in U(2^n) \mid u^\dagger P u \in \mathcal{P}_n, \ \forall P \in \mathcal{P}_n\} = \mathcal{N}_u(\mathcal{P}_n)$$

These include important gates such as H, CX, S, which we can think of as living one step above the Paulis. It has been shown that all circuits composed of $\mathcal{C}_2$ gates (and classical states, e.g. $|0\rangle \otimes \cdots \otimes |0\rangle$) are **efficiently classically simulable** with the stabilizer formalism [1, 2].

But if Quantum Advantage exists, there must be more to the story than circuits built out of $\mathcal{C}_2$ gates. Following the same pattern of conjugation taking us one step down, we can define a hierarchy of gatesets which will be relevant in the next section.

Definition 1.1. ***Level-k of the Clifford Hierarchy:*** $\mathcal{C}_k = \{u \in U(2^n) \mid u^\dagger P u \in \mathcal{C}_{k-1}, \ \forall P \in \mathcal{C}_{k-1}\}$

1.2 Universal Gatesets

In classical computing, a **universal gateset** is a finite collection of gates from which all others can be built i.e. all circuits can be decomposed in terms of logical operators in the gateset. In the classical setting, the NAND gate alone is sufficient!

In Quantum Computing, because logical gates on n-Qubits correspond to Unitary Matrices $u \in U(2^n)$, the definition must be generalized.

Definition 1.2. ***Universal Gateset:*** *A finite set of gates $\mathcal{G}$ is said to be universal if any $u \in U(2^n)$ can be approximated arbitrarily well using finitely many words generated by the members of $\mathcal{G}$. Note: for $k \geq 3$, these are often referred to as* non-cliffords *to distinguish*

from the cliffords i.e. $\mathcal{C}_2$.

Note. *The proof that there exist universal gatesets for quantum computation was given independently by Solovay and Kitaev, and is now known as the Solovay-Kitaev Theorem.*

It is essentially a proof by construction (see [3] for a pedagogical view) showing, for a given unitary U and desired accuracy ε, how to construct a sequence S of of gates from the gateset $\mathcal{G}$ of length $O(log^c(1/\varepsilon))$ such that $d(U,S) < \varepsilon$.

There have been many improvements and optimizations over the years giving better compilation algorithms.

It turns out that

$$\{\text{Set of gates generating } \mathcal{C}_2\} + (\text{anything else higher in the Clifford Hierarchy})$$

is sufficient for universal quantum computation. Since the cliffords are classically simulable, it is the non-clifford gate that allows us to achieve universal quantum computation and so the non-clifford part of our gateset is, somewhat dramatically, referred to as

Thus, implementing these magic gates is vital to making a scalable quantum computer.

One of the most commonly pursured universal gate sets is $\{\text{Cliffords }\} + T$ where $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$

Is there a reason the T gate is preferred?

2 Magic in Codes

2.1 Fault-Tolerance and Transversality

In practice, on a quantum computer, we are often restricted to a finite set of "small operations" due to the hardware difficulty of non-local operations and the fact that more operations = more chances for errors.

We want our physical implementations of logical gates to be **"Fault-Tolerant"**, which roughly means they "do not spread errors (too badly)". A slightly more quantitative way to state this is, given the set of correctable errors within the code $\mathcal{E}_{corr}$ and a physical implementation of logical operator $\overline{U}$, we want

$$\overline{U}\mathcal{E}_{corr}\overline{U}^\dagger \approx \mathcal{E}_{corr}$$

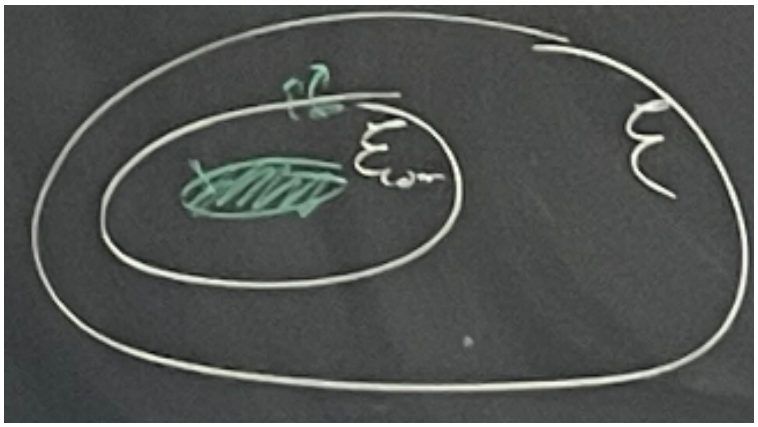


Figure 1: Fault tolerance means the vast majority of correctable errors *remain* correctable after a logical gate is applied, although we allow for a little bit of error to spread, which may be caught by stabilizer measurements.

Definition 2.1. *Transversal Gate We say a logical operator $\overline{U}$ has a transversal implementation if it can be implemented using a collection of independent physical gates acting on distinct blocks of qubits i.e. we have*

$$\overline{U} = \bigotimes_{i=1}^{|B|} U_i$$

for some set of qubit blocks B and corresponding physical gates.

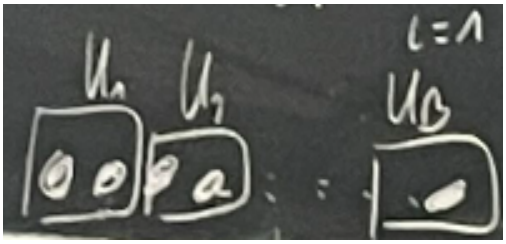


Figure 2: Independent gates $U_1, \cdots, U_{|B|}$ act on each of the blocks of qubits in a transversal implementation.

Transversality guarantees fault-tolerance because it is impossible for errors to spread between distinct blocks, and is thus a very desirable property for *all* logical operators to have. Unfortunately life is not so simple, and there are some very powerful No-Go theorems that limit our capability for magic in codes. Namely,

Theorem 2.1. ***(Eastin-Knill):*** *No code that detects a single qubit error can have a* ***universal gate set*** *which is also* ***transversal*** *[4].*

and

Theorem 2.2. ***(Bravyi-König):*** *In a geometrically local Pauli Stabilizer code in D-dimensions, locality preserving gates can only implement logical operators in $\mathcal{C}_D$ on the state(s) encoded by the code [5].*

In particular, that means that in 2D (which is where many of the most important QECCs live), we can only implement $\mathcal{C}_2$ i.e. clifford gates transversally.

Note. *The original Bravyi-König paper is a slightly tough read; one can find a more approachable argument in [6].*

Further, there are results on the combinatorial structures required in order to transversally implement specific types of unitaries .

ask Julio for details/refs

So then, how do we go about achieving magic in codes? Let's discuss two (relatively) well studied approaches.

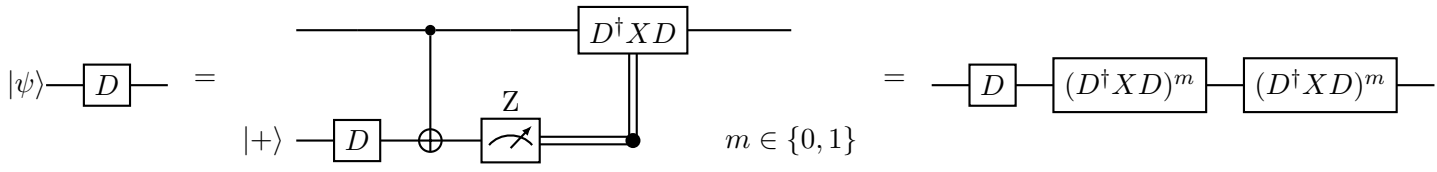
2.2 Gate Injection

If we are trying to implement Diagonal non-clifford gates i.e. $u \in \text{diag}(\mathcal{C}_3)$, such as

- The T gate, $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$
- Controlled-S, $C_iS_j = |0\rangle\langle 0|_i \otimes \mathbb{1}_j + |1\rangle\langle 1|_i \otimes S_j$
- Controlled-Controlled-Z, $C_iC_jC_k = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \end{pmatrix}$

we can get around the No-Go theorems by *precomputing/preparing* some resource states and *injecting them* into the circuit when we need to apply the correspond gate, thus using up the resource state.

In particular, for diagonal non-clifford D let's denote $|D\rangle := D|+\rangle$. Then, we have the following circuit equality:



Let's get some intuition for what's happening here by considering $D = T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$. In the bottom-left, we have the pre-prepared resource state $|T\rangle = T|+\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + e^{i\pi/4}|1\rangle\right)$. The CNOT is controlled on $|\psi\rangle$ and the resource state is the Target.

For data state $|\psi\rangle = a|0\rangle + b|1\rangle$, the desired output state is $T|\psi\rangle = a|0\rangle + be^{i\pi/4}|1\rangle$.

Just before the CNOT, the overall wavefunction looks like

$$|\psi\rangle \otimes |T\rangle = \frac{1}{\sqrt{2}}\left[a|00\rangle + ae^{i\pi/4}|01\rangle + b|10\rangle + be^{i\pi/4}|11\rangle\right]$$

So after the CNOT gate, we have

$$\text{CNOT}\left(|\psi\rangle \otimes |T\rangle\right) = \frac{1}{\sqrt{2}}\left[a|00\rangle + ae^{i\pi/4}|01\rangle + b|11\rangle + be^{i\pi/4}|10\rangle\right]$$

Now, measuring out the resource state qubit in the Z basis, we get a measurement $m \in \{0, 1\}$ corresponding to the state of the qubit.

- If $m = 0$, we've collapsed to the state $|\psi'\rangle = a|0\rangle + be^{i\pi/4}|1\rangle$ **which is what we wanted!**
- If $m = 1$, we've collapsed onto the state $|\psi'\rangle = ae^{i\pi/4}|0\rangle + b|1\rangle$, so we need to apply a correction to get the desired $T|\psi\rangle$ state. Namely,

$$\begin{aligned} & T|\psi'\rangle = ae^{i\pi/4}|0\rangle + be^{i\pi/4}|1\rangle \\ \implies & XT|\psi'\rangle = ae^{i\pi/4}|1\rangle + be^{i\pi/4}|0\rangle \\ \implies & T^{-1}XT|\psi'\rangle = a|1\rangle + be^{i\pi/4}|0\rangle \\ \implies & X\left(T^{-1}XT\right)|\psi'\rangle = a|0\rangle + be^{i\pi/4}|1\rangle \end{aligned}$$

which is what we wanted! Notice that X is a Pauli gate and, crucially, $(T^{-1}XT)$ is a **Clifford** since $T \in \mathcal{C}_3$.

The point here is that even if $m = 1$ and we don't immediately have the desired magic state, we can apply a sequence of **Clifford** gates to get what we need.

Note. *This is slightly different from the usual* ***Gate Teleportation*** *protocol, wherein the order of the CNOT is reversed i.e. the resource/ancilla state is the control and the data qubit is the target, the data qubit gets measured out and the logical information transferred over to the ancilla after measurement.*

2.3 Common ways to prepare Magic

Magic State Distillation

A very popular approach to obtaining high fidelity $|D\rangle$ states is to start off with many erroneous or low-quality $|D\rangle$ states and **"distill"** them down to fewer, higher-quality states.
This is a probabilistic scheme, but we can simply chain/concatenate more and more levels/layers of distillation to get better and better fidelities. At each level, the distillation circuit involved is comprised of Cliffords, making it feesible to simulate and implement fault-tolerantly.

Distillation gives us a fault-tolerant way to produce magic, but it is extremely resource/overhead heavy. add more detail

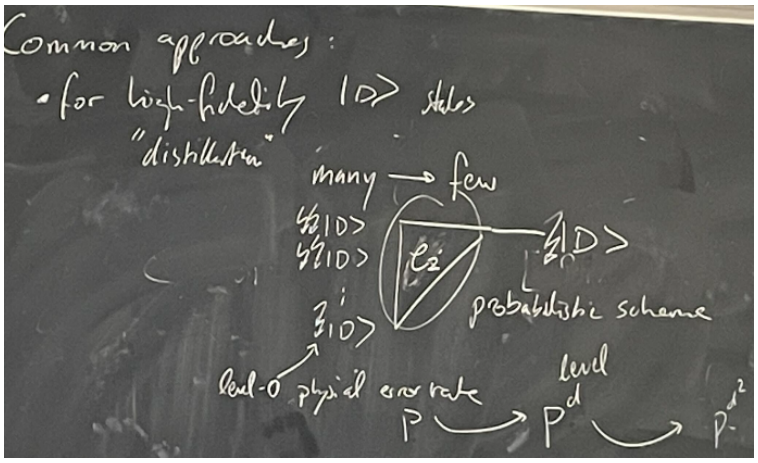


Figure 3: Magic State Distillation

Code Switching

Different codes may have different sets of gates they can implement transversally, and so one approach to preparing magic states is to do most of our computation in one code but temporarily, whenever needed, switch into some other code within which magic is easier to prepare.

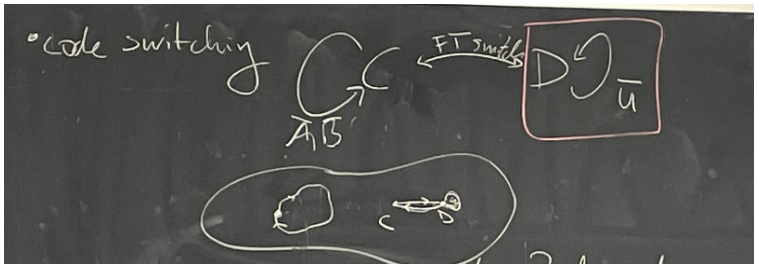


Figure 4: (Upper figure) We can imagine that codes C and D respectively have transversal implementations of $\{\overline{A}, \overline{B}\}$ and $\overline{U}$, where $\overline{U}$ is some magic gate. We could work primarily in C for the majority of the computation where we simply need cliffords, but Fault-Tolerantly switch into the D code when we need to use the $\overline{U}$ gate and then switch back. (Bottom) Any computation can be thought of as a closed-loop in an abstract state space, where we have a closed loop because we remain in the same codespace. Code Switching can be viewed as composing these paths to form a closed loop.

Julio had mentioned there's a lot of beautiful math underlying these ideas. Ask him for references.

3 New kid on the block: Non-Pauli Codes

Non-Pauli Codes

Recently there have been many exciting developments in the usage of non-Pauli codes. These are codes whose stabilizers are composed of not only pauli gates, but also clifford gates.

Eg. Preparing $|T\rangle$ state

Note that the $|T\rangle = T|+\rangle$ state is an eigenstate of the clifford operator $XS \in \mathcal{C}_2$. Thus, within the non-Pauli code, the *encoded/logical* $|\overline{T}\rangle$ state is the (unique) eigenstate of the stabilizers generated by $\langle \overline{XS}, \{S_i\}_i \rangle$ where $\{S_i\}_i$ are, say, stabilizers of a Pauli code. We can enforce $XS = \pm 1$ by measurement i.e. project onto the eigenspace giving us $\pm|T\rangle$.

The problem now is, **how do we implement a $\overline{XS}$ stabilizer fault-tolerantly?**

Fault-Tolerance

It turns out we need a **"weight-reduction"** that uses some extra qubits. The approach we take is to use these extra qubits and some entangling gates to deform the stabilizers $S = \{S_i\}_i$ into a set of stabilizers $\tilde{S} = \{\tilde{s}_i\}_i$ such that $\overline{XS} \in \langle \{\tilde{s}_i\}_i \rangle$ i.e. $\overline{XS}$ is generated by the deformed stabilizers. This way, carrying out the deformed stabilizer measurements lets us implicitly measure $\overline{XS}$.

After staying in the non-Pauli code for a while (we stay in it for time linear w.r.t the desired fault-distance of the protocol), we return to the original Pauli code. The resulting protocol looks like:

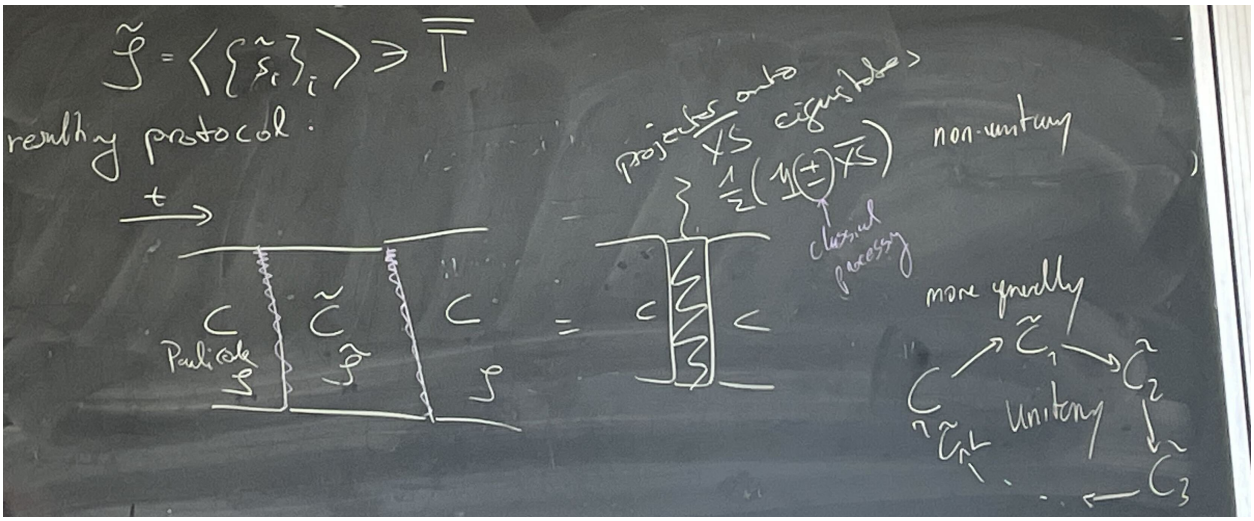


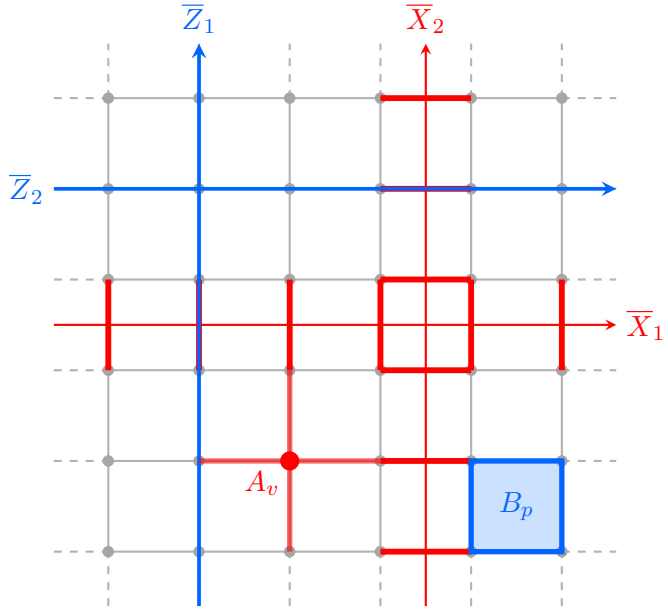
Figure 5: **(Left)** We deform from the original stabilizer group S by entangling with the extra qubits to get deformed stabilizer group $\tilde{S}$ and then measure out the extra qubits in Z basis to return to S. **(Right)** We can chain a bunch of different codes to implement different logical unitaries.

To get a better feel for things, in the next section we wil see and example of preparing a diagonal clifford logical within a CSS code.

Julio mentioned something about the procedure shown on bottom right and how it hasn't yet been generalized to QLDPC codes. Ask about

4 Algebra of Twisted CSS Codes (By Example)

Consider two Toric Codes overlapped with each other (say, with two types of qubits Red and Blue on each edge of the lattice).



Recall that a single lattice has a chain complex associated with it

$$C_0 \xrightarrow{d} C_1 \xrightarrow{d} C_2$$

where C_i is the set of i-chains.

This induces another chain complex

$$\left(\mathbb{Z}_2^{C_0} \xrightarrow{d} \mathbb{Z}_2^{C_1} \xrightarrow{d} \mathbb{Z}_2^{C_2}\right)^{\oplus 2}$$

$$S_X \longleftarrow \text{qubits} \longrightarrow S_Z$$

This code with two copies of the Toric Code has a logical $\overline{CZ}$ implmentation as

$$\overline{CZ} = \bigotimes_{f \in C_2} f$$

We "weight-reduce" $\overline{CZ}$ by adding Green qubits to every edge and defining new stabilizers:

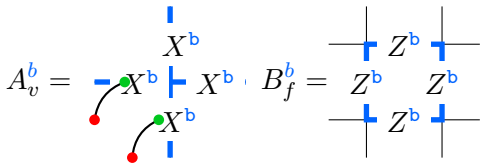
$$A_v^g = \quad = CZ \times CZ \prod_{i \in dv} X_i^g, \qquad B_f^g =$$

where dv is the 1-chain formed by the edges of which v is a vertex.

Note that $\prod_{v \in C_0} A_v^g = \overline{CZ}$ so adding these into our stabilizer generator set means we implicitly measure $\overline{CZ}$.

In order to commute with these green stabilizers on a shared $+1$ eigenspace, the red and blue vertex stabilizers are *deformed* (the plaquette stabilizers remain the same):

$$A_v^r = \quad , \qquad B_f^r = \quad ,$$



and our new stabilizer generator set is $\tilde{S} = \langle \{A_v^c, B_f^c\}_{v,f,c} \rangle$.

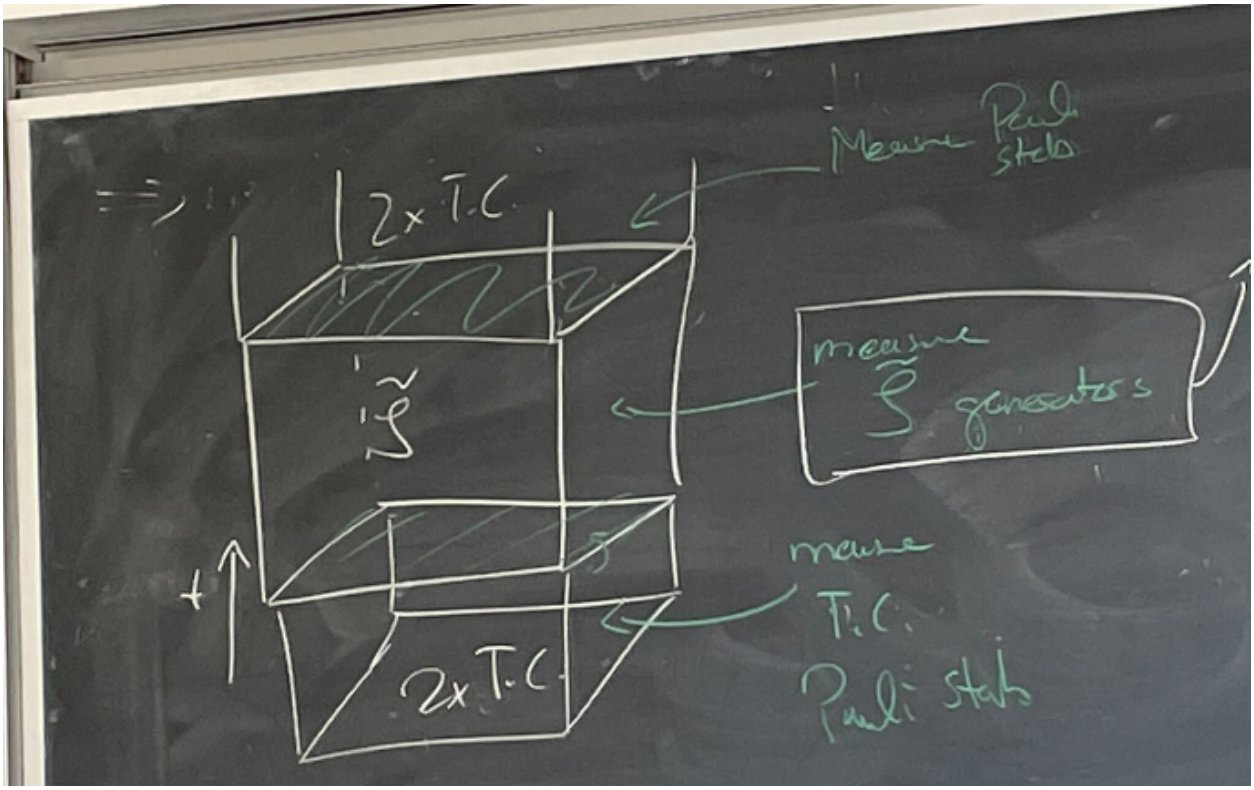


Figure 6: **Spacetime diagram of the procedure (with time flowing upwards):** We start off with the two Toric Code copies and carry out the usual Toric Code Pauli stabilizer measurements. Then, when we want to prepare a $|T\rangle$ state we introduce the green qubits and entangle them with the red and green qubits by measuring A_v^g, B_f^g, after which we continue measuring the deformed red and blue stabilizers. After a few rounds, we measure out the green qubits in the Z basis to return to the two Toric Code copies and continue measuring Pauli stabilizers.

In order to measure the $\tilde{S}$ generators, we must measure the A_v^c, B_f^c stabilizers. However, since they do not all commute with each other outside of the $+1$ eigenstate we require **"Just-In-Time decoding"** This is not necessary in Clifford Circuits/Pauli Syndrome Extraction because commuting measurements are independent – $\boldsymbol{A_v}$ **and** $\boldsymbol{B_p}$ **are not!**

Commutation Relations of Stabilizers

More details can be found in [7] but roughly speaking, the group commutator $[A, B] = ABA^{-1}B{-}1$ for the vertex stabilizers as a result is not trivial but rather of the form

$$[A_v^c, A_{v'}^{c'}] \in \langle B_p^c \rangle \in \mathcal{S}^Z$$

where $\mathcal{S}^Z$ is the subgroup of stabilizers generated by the Plaquette stabilizers (which are all of Pauli-Z type). So, in the presence of plaquette violations, the vertex stabilizers anticommute and we are taken out of the codespace formed by the all-(+1)-eigenspace.

Similarly,

$$[P^X, A_v^c] \in P^Z$$

where $P^X(P^Z)$ are Pauli X(Z) type operators. So, X errors cause "residual" Z errors because their commutators with vertex stabilizers give Pauli-Z products.

Generally, the deformed stabilizer groups for such protocols have the form $\tilde{S} = \langle \{D_i P_i^x, P^z\}, \quad D_i \in \mathcal{C}_2 \text{ and diagonal} \rangle$

The take-home message is that the protocol only works if X errors are sparse in the circuit.

5 Future directions

Some directions for work required to flesh out this twisted approach to magic state preparation are:

1. **New constructions and simulations:**
   - New constructions are desired in order to optimize/minimize the spacetime overhead required for these protocols.
   - Numerical simulations concretely benchmarking these magic-state-preparation methods and comparing them quantitatively to approaches like Distillation and Cultivation are required. A major hurdle to such projects has been the difficulty in simulating circuits with (physical) non-clifford gates, however there have been a number of recent results that open this direction.
2. **Threshold theorems under local stochastic noise:**
   - Ask Julio for some more context. Not sure what exactly the current major problems are.

References

[1] Daniel Gottesman. Stabilizer codes and quantum error correction, 1997.

[2] Daniel Gottesman. The heisenberg representation of quantum computers, 1998.

[3] Christopher M. Dawson and Michael A. Nielsen. The solovay-kitaev algorithm, 2005.

[4] Bryan Eastin and Emanuel Knill. Restrictions on transversal encoded quantum gate sets. *Physical review letters*, 102(11):110502, 2009.

[5] Sergey Bravyi and Robert König. Classification of topologically protected gates for local stabilizer codes. *Physical review letters*, 110(17):170503, 2013.

[6] Fernando Pastawski and Beni Yoshida. Fault-tolerant logical gates in quantum error-correcting codes. *Physical Review A*, 91(1):012305, 2015.

[7] Margarita Davydova, Andreas Bauer, Julio C. Magdalena de la Fuente, Mark Webster, Dominic J. Williamson, and Benjamin J. Brown. Universal fault tolerant quantum computation in 2d without getting tied in knots, 2025.